

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 1

Disciplina: Tehnici de programare

Tema: ” Python ”

A efectuat :

st. gr.SI – 261 Iurco Mihai

A verificat :

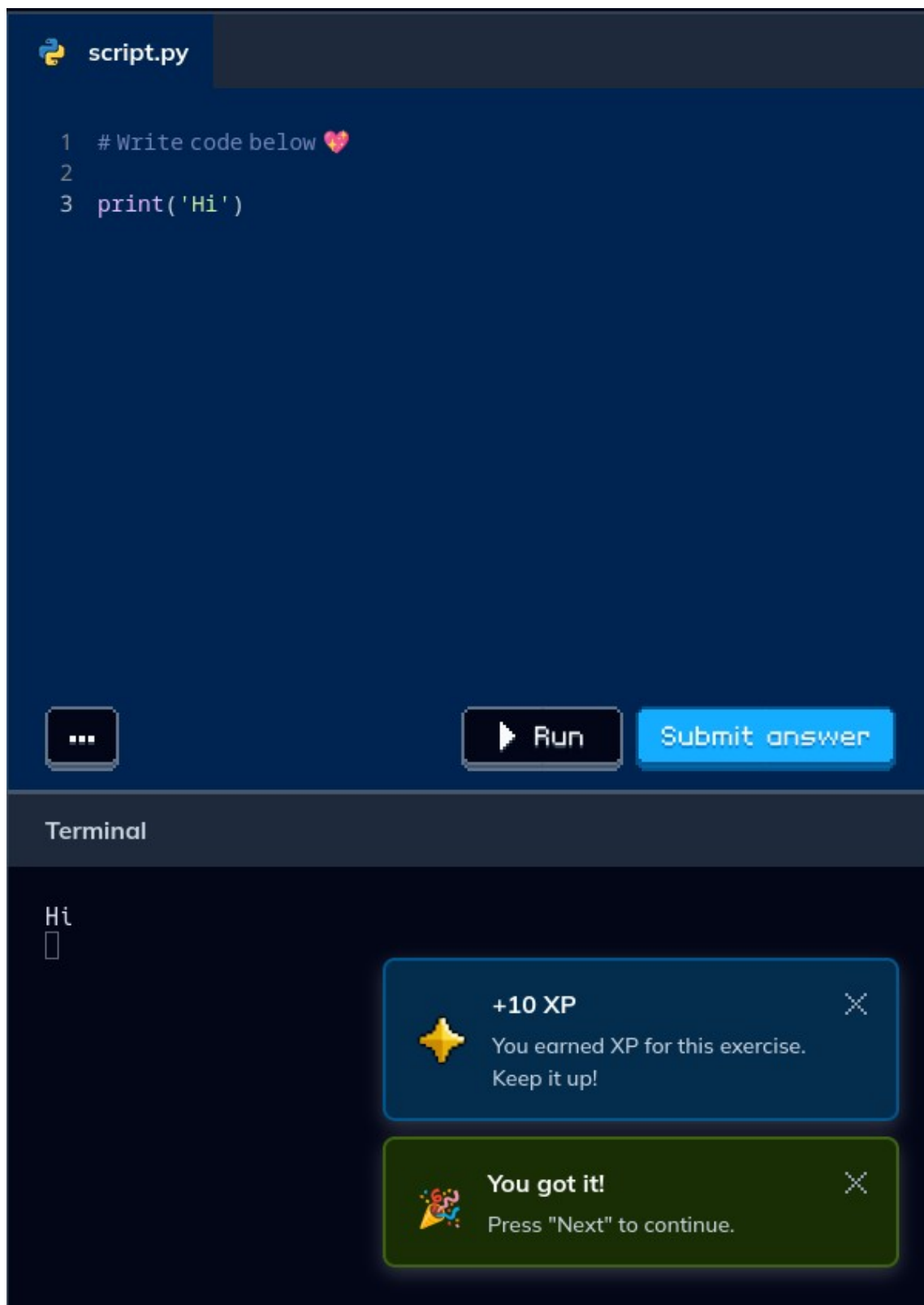
asistent universitar XXXXXX XXXX

Chișinău – 2026

SARCINA LUCRĂRII

1. codedex.io - Hello World:

1. Setting Up:



2. Hello World:

 **hello_world.py**

```
1 # Write code below ❤️
2
3 print('Hello World')
```

 **Run** **Submit answer**

Terminal

```
Hello World
█
```

 **+10 XP** 
You earned XP for this exercise.
Keep it up!

 **You got it!** 
Press "Next" to continue.

3. Pattern:

 pattern.py

```
1 # Write code below ❤️
2 print('  1')
3 print(' 2 3')
4 print('4 5 6')
5 print('7 8 9 10')
```

 Run 

Terminal

1
2 3
4 5 6
7 8 9 10
□

 **+10 XP** 
You earned XP for this exercise.
Keep it up!

 **You got it!** 
Press "Next" to continue.

4. Initials:

 initials.py

```
1 # Write code below ❤️
2
3 print("DDDD L")
4 print("D D L")
5 print("D D L")
6 print("D D L")
7 print("D D L")
8 print("D D L")
9 print("DDDD LLLLL")
```

 Run  Try again in 6s

Terminal

```
DDDD L
D D L
D D L
D D L
D D L
D D L
DDDD LLLLL
█
```

 **+15 XP** 
You earned XP for this exercise.
Keep it up!

 **You got it!** 
Press "Next" to continue.

5. Snail Mail:

 letter.py

```
1 # Write code below ❤️
2
3 print("22.09.2026")
4 print("Indifferent")
5 print("Forging a career for myself")
6 print("May we live a wonderfull future")
7 print("💀")
```

 Run  Try again in 4s

Terminal

```
22.09.2026
Indifferent
Forging a career for myself
May we live a wonderfull
💀
█
```

 **+15 XP** 
You earned XP for this exercise.
Keep it up!

 **You got it!** 
Press "Next" to continue.

2. codedex.io – Variables:

1. Data Types:

 **script.py**

```
1 # Write code below ❤️
2
3 name = 'Iurco Mihai'
4 age = 20
5 pi = 3.14
6 verified = True
7
8 print(name)
9 print(age)
10 print(pi)
11 print(verified)
```

 **Run** **Submit answer**

Terminal

```
Iurco Mihai
20
3.14
True
█
```

 **+15 XP** 
You earned XP for this exercise.
Keep it up!

 **You got it!** 
Press "Next" to continue.

2. Temperature:

 temperature.py

```
1 # Write code below ❤️
2
3 Fahrenheit = 60
4 Celsius = (Fahrenheit - 32)/1.8
5 print(Celsius)
```

 Run  Try again in 3s

Terminal

15.555555555555555
█

 **+15 XP** 
You earned XP for this exercise.
Keep it up!

 **You got it!** 
Press "Next" to continue.

3. BMI:

 **bmi.py**

```
1 # Write code below ❤️
2 mass = 85
3 height = 187
4 bmi = mass/pow(height, 2)
5 print(bmi)
```

 **Run** Try again in 4s

Terminal

```
0.0024307243558580457
█
```

 **+15 XP** 
You earned XP for this exercise.
Keep it up!

 **You got it!** 
Press "Next" to continue.

4. Pythagorean:

 **hypotenuse.py**

```
1 # Write code below ❤️
2 import math
3 a = int(input("Inser a: "))
4 b = int(input("Inser b: "))
5
6 c = math.sqrt(pow(a, 2) + pow(b, 2))
7 print("The Pythagorean theorem: ", c)
```

 **Run** Try again in 4s

Terminal

```
Inser a: 8
Inser b: 2
The Pythagorean theorem:
█
```

 **+15 XP** 
You earned XP for this exercise.
Keep it up!

 **You got it!** 
Press "Next" to continue.

5. Currency:

 **currency.py**

```
1 # Write code below ❤️
2
3 pesos = int(input("What do you have left in pesos?: "))
4 soles = int(input("What do you have left in soles?: "))
5 reais = int(input("What do you have left in reais?: "))
6
7 Total = (pesos * 0.00032 + soles * 0.30 + reais * 0.20)
8 print("The total you have in USD is: ", Total)
```

 **Run**  Try again in 5s

Terminal

```
What do you have left in pesos?: 5600
What do you have left in soles?: 105
What do you have left in reais?: 
The total you have in USD is: 89
█
```

 **+20 XP** 
You earned XP for this exercise.
Keep it up!

 **You got it!** 
Press "Next" to continue.

3. learnpython.org – Welcome:

1. Hello, World!:

script.py solution.py

1 print("Hello, World!")

IPython Shell

<script.py> output:
Goodbye, World!

<script.py> output:
Hello, World

<script.py> output:
Hello, World!

In [1]: |

Great job!

Submit ●

2. Variables and Types:

script.py solution.py

1 # change this code
2 mystring = "hello"
3 myfloat = 10.0
4 myint = 20
5
6 # testing code
7 if mystring == "hello":
8 print("String: %s" % mystring
9)
9 if isinstance(myfloat, float) and
myfloat == 10.0:
10 print("Float: %f" % myfloat)
11 if isinstance(myint, int) and
myint == 20:
12 print("Integer: %d" % myint)

IPython Shell

<script.py> output:
String: hello
Float: 10.000000
Integer: 20

In [1]: |

Great job!

Submit ●

3. Lists:

script.py	solution.py	IPython Shell
<pre>4 5 # write your code here 6 numbers.append(1) 7 numbers.append(2) 8 numbers.append(3) 9 10 strings.append("hello") 11 strings.append("world") 12 13 second_name = names[1] 14 15 # this code should write out the filled arrays and the second name in the names list (Eric). 16 print(numbers) 17 print(strings)</pre>		<script.py> output: <pre>[1, 2, 3] ['hello', 'world'] The second name on the names list is Eric</pre> In [1]:
Great Job!		x
Submit ●		🔍

4. Basic Operators:

script.py	solution.py	IPython Shell
<pre>5 x_list = [x] * 10 6 y_list = [y] * 10 7 big_list = x_list + y_list 8 9 print("x_list contains %d objects" % len(x_list)) 10 print("y_list contains %d objects" % len(y_list)) 11 print("big_list contains %d objects" % len(big_list)) 12 13 # testing code 14 if x_list.count(x) == 10 and y_list.count(y) == 10: 15 print("Almost there...") 16 if big_list.count(x) == 10 and</pre>		<script.py> output: <pre>x_list contains 10 objects y_list contains 10 objects big_list contains 20 objects Almost there... Great!</pre> In [1]:
Good work!		x
Submit ●		🔍

5. String Formatting:

script.py	solution.py	IPython Shell
<pre>1 data = ("John", "Doe", 53.44) 2 format_string = "Hello %s %s. Your current balance is %s." 3 4 print(format_string % data)</pre>		<script.py> output: Hello John Doe. Your current balance is \$53.44. In [1]:
Great work!		
Submit		

6. Basic String Operations:

script.py	solution.py	IPython Shell
<pre>22 print("String in lowercase: %s" % s.lower()) 23 24 # Check how a string starts 25 if s.startswith("Str"): 26 print("String starts with 'Str'. Good!") 27 28 # Check how a string ends 29 if s.endswith("ome!"): 30 print("String ends with 'ome!'. Good!") 31 32 # Split the string into three separate strings, 33 # each containing only a word 34 print("Split the words of the string: %s" % s.split(" "))</pre>		The first five characters are 'Strin' The next five characters are 'gs ar' The thirteenth character is 'a' The characters with odd index are 'tig r wsm!' The last five characters are 'some!' String in uppercase: STRINGS ARE AWESOME! String in lowercase: strings are awesome! String starts with 'Str'. Good! String ends with 'ome!'. Good! Split the words of the string: ['Strings', 'are', 'awesome!'] In [1]:
Submit		

7. Conditions:

script.py	solution.py	IPython Shell
<pre>1 # change this code 2 number = 16 3 second_number = 0 4 first_array = [1,2,3] 5 second_array = [1,2] 6 7 if number > 15: 8 print("1") 9 10 if first_array: 11 print("2") 12 13 if len(second_array) == 2: 14 print("3") 15 16 if len(first_array) + len</pre>		<script.py> output: <pre>1 2 3 4 5 6</pre> In [1]:
Great Work! ✕		
Submit ● 🔗		

8. Loops:

script.py	solution.py	IPython Shell
<pre>1 numbers = [2 951, 402, 984, 651, 360, 69, 3 408, 319, 601, 485, 980, 507, 725 4 , 547, 544, 5 615, 83, 165, 141, 501, 263, 6 617, 865, 575, 219, 390, 984, 592 7 , 236, 105, 942, 941, 8 386, 462, 47, 418, 907, 344, 9 236, 375, 823, 566, 597, 978, 328 10 , 615, 953, 345, 11 399, 162, 758, 219, 918, 237, 12 412, 566, 826, 248, 866, 950, 626 13 , 949, 687, 217, 14 815, 67, 104, 58, 512, 24, 15 892, 894, 767, 553, 81, 379, 843, 16 831, 445, 742, 717, 17 858, 660, 940, 151, 680, 758</pre>		<pre>592 236 942 386 462 418 344 236 566 978 328 162 758 918</pre>
Great work! ✕		
Submit ● 🔗		

9. Functions:

script.py	solution.py	IPython Shell
<pre>5 # Modify this function to concatenate to each benefit - " is a benefit of functions!" 6 def build_sentence(benefit): 7 return "%s is a benefit of functions!" % benefit 8 9 10 def name_the_benefits_of_function s(): 11 list_of_benefits = list_benefits() 12 for benefit in list_of_benefits: 13 print(build_sentence (benefit))</pre>		<script.py> output: <pre>More organized code is a benefit of functions! More readable code is a benefit of functions! Easier code reuse is a benefit of functions! Allowing programmers to share and connect code together is a benefit of functions!</pre> In [1]:
Nice work! ✕		
Submit ● 		

10. Classes and Objects:

script.py	solution.py	IPython Shell
<pre>9 return desc_str 10 11 # your code goes here 12 car1 = Vehicle() 13 car1.name = "Fer" 14 car1.color = "red" 15 car1.kind = "convertible" 16 car1.value = 60000.00 17 18 car2 = Vehicle() 19 car2.name = "Jump" 20 car2.color = "blue" 21 car2.kind = "van" 22 car2.value = 10000.00 23 24 # test code</pre>		<script.py> output: <pre>Fer is a red convertible worth \$60000 .00. Jump is a blue van worth \$10000.00.</pre> In [1]:
Great job! ✕		
Submit ● 		

11. Dictionaries:

script.py	solution.py	IPython Shell
<pre>1 phonebook = { 2 "John" : 938477566, 3 "Jack" : 938377264, 4 "Jill" : 947662781 5 } 6 7 # your code goes here 8 phonebook["Jake"] = 938273443 9 del phonebook["Jill"] 10 11 # testing code 12 if "Jake" in phonebook: 13 print("Jake is listed in the 14 phonebook.") 15 if "Jill" not in phonebook:</pre>		<script.py> output: <pre>Jake is listed in the phonebook. Jill is not listed in the phonebook.</pre> In [1]:
Nice work! ✕		
Submit ● 		

12. Modules and Packages:

script.py	solution.py	IPython Shell
<pre>1 import re 2 3 # Your code goes here 4 find_members = [] 5 for member in dir(re): 6 if "find" in member: 7 find_members.append 8 (member) 9 print(sorted(find_members))</pre>		<script.py> output: <pre>['findall', 'finditer']</pre> In [1]:
Great work! ✕		
Submit ● 		

4. .learnpython.org – Advanced Tutorials:

1. Generators:

script.py	solution.py	IPython Shell
<pre>1 # fill in this function 2 def fib(): 3 a, b = 1, 1 4 while 1: 5 yield a 6 a, b = b, a + b 7 8 # testing code 9 import types 10 if type(fib()) == types .GeneratorType: 11 print("Good, The fib function is a generator.") 12 13 counter = 0 14 for n in fib():</pre>		<script.py> output: <pre>Good, The fib function is a generator. 1 1 2 3 5 8 13 21 34 55</pre>
Good work! ✕ Submit ●		

2. List Comprehensions:

script.py	solution.py	IPython Shell
<pre>1 numbers = [34.6, -203.4, 44.9, 68 .3, -12.2, 44.6, 12.7] 2 newlist = [int(x) for x in numbers if x > 0] 3 print(newlist)</pre>		<script.py> output: <pre>[34, 44, 68, 44, 12]</pre> In [1]:
Very nice! ✕ Submit ●		

3. Lambda Functions:

script.py	solution.py	IPython Shell
<pre>1 l = [2,4,7,3,14,19] 2 for i in l: 3 # your code here 4 my_lambda = lambda x : (x % 2 5) == 1 6 print(my_lambda(i))</pre>		<script.py> output: <pre>False False True True False True</pre> In [1]:
Nice work!		
Submit 		

4. Multiple Function Arguments:

script.py	solution.py	IPython Shell
<pre>1 # edit the functions prototype and implementation 2 def foo(a, b, c, *args): 3 return len(args) 4 5 def bar(a, b, c, **kwargs): 6 return kwargs["magicnumber"] 7 == 7 8 9 # test code 10 if foo(1, 2, 3, 4) == 1: 11 print("Good.") 12 if foo(1, 2, 3, 4, 5) == 2: 13 print("Better.") 14 if bar(1, 2, 3, magicnumber=6) ==</pre>		<script.py> output: <pre>Good. Better. Great. Awesome!</pre> In [1]:
Great work!		
Submit 		

5. Regular Expressions:

script.py	solution.py	IPython Shell
<pre>(your_pattern) 5 emails = ["john@example.com", "python-list@python.org", "what .lan?ug{}ly@email.com"] 6 for email in emails: 7 if not re.match(pattern, email): 8 print("You failed to match %s" % (email)) 9 elif not your_pattern: 10 print("Forgot to enter a pattern!") 11 else: 12 print("Pass") 13 # Your pattern here! 14 pattern = r"\"?([-a-zA-Z0-9.`?{}]</pre>		<pre><script.py> output: Pass Pass Pass In [1]: </pre>
Great work! ✕		
Submit ●		

6. Exception Handling:

script.py	solution.py	IPython Shell
<pre>1 actor = {"name": "John Cleese", "rank": "awesome"} 2 3 def get_last_name(): 4 return actor["name"].split 5 ()[1] 6 get_last_name() 7 print("All exceptions caught! Good job!") 8 print("The actor's last name is %s" % get_last_name())</pre>		<pre><script.py> output: All exceptions caught! Good job! The actor's last name is Cleese In [1]: </pre>
Great work! ✕		
Submit ●		

7. Sets:

script.py	solution.py	IPython Shell
<pre>1 a = ["Jake", "John", "Eric"] 2 b = ["John", "Jill"] 3 4 A = set(a) 5 B = set(b) 6 7 print(A.difference(B))</pre>		<script.py> output: {'Jake', 'Eric'} In [1]:
Nice work! ✕		
Submit ● 🔗		

8. Serialization:

script.py	solution.py	IPython Shell
<pre>5 def add_employee(salaries_json, 6 name, salary): 7 salaries = json.loads 8 (salaries_json) 9 salaries[name] = salary 10 11 return json.dumps(salaries) 12 13 # test code 14 salaries = '{"Alfred" : 300, 15 "Jane" : 400 }' 16 new_salaries = add_employee 17 (salaries, "Me", 800) 18 decoded_salaries = json.loads 19 (new_salaries) 20 print(decoded_salaries["Alfred"])</pre>		<script.py> output: 300 400 800 In [1]:
Great work! ✕		
Submit ● 🔗		

9. Partial Functions:

script.py	solution.py	IPython Shell
<pre>1 from functools import partial 2 def func(u, v, w, x): 3 return u*4 + v*3 + w*2 + x 4 5 p = partial(func,5,6,7) 6 print(p(8))</pre>		<script.py> output: 60 In [1]:
Good job!		
Submit		

10. Code Introspection:

script.py	solution.py	IPython Shell
<pre>1 # Define the Vehicle class 2 class Vehicle: 3 name = "" 4 kind = "car" 5 color = "" 6 value = 100.00 7 def description(self): 8 desc_str = "%s is a %s %s 9 worth %.2f." % (self.name, self 10 .color, self.kind, self.value) 11 return desc_str 12 # Print a list of all attributes 13 # of the Vehicle class. 14 print(dir(Vehicle))</pre>		<script.py> output: ['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__gt__', '__hash__', '__init__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'color', 'description', 'kind', 'name', 'value']
Very nice!		
Submit		

11. Closures:

script.py	solution.py	IPython Shell
<pre>1 def multiplier_of(n): 2 def multiplier(number): 3 return number*n 4 return multiplier 5 6 multiplywith5 = multiplier_of(5) 7 print(multiplywith5(9))</pre>		<script.py> output: 45 In [1]:
Great work!		
Submit		

12. Decorators:

script.py	solution.py	IPython Shell
<pre>1 def type_check(correct_type): 2 def check(old_function): 3 def new_function(arg): 4 if isinstance(arg, 5 correct_type)): 6 return 7 old_function(arg) 8 else: 9 print("Bad Type") 10 return new_function 11 return check 12 13 @type_check(int) 14 def times2(num): 15 return num*2</pre>		<script.py> output: 4 Bad Type H Bad Type In [1]:
Good job!		
Submit		

13. Map, Filter, Reduce:

script.py	solution.py	IPython Shell
<pre>1 ##### Map 2 from functools import reduce 3 4 my_floats = [4.35, 6.09, 3.25, 9 5 .77, 2.16, 8.88, 4.59] 6 my_names = ["olumide", "akinremi", 7 "josiah", "temidayo", 8 "omoseun"] 9 my_numbers = [4, 6, 9, 23, 5] 10 11 map_result = list(map(lambda x: 12 round(x ** 2, 3), my_floats)) 13 filter_result = list(filter(14 lambda name: len(name) <= 7, 15 my_names)) 16 reduce_result = reduce(lambda</pre>		<script.py> output: <pre>[18.922, 37.088, 10.562, 95.453, 4.666 , 78.854, 21.068] ['olumide', 'josiah', 'omoseun'] 24840</pre> In [1]:
Congrats! Nice work. ✕		
Submit ● 🔗		

CONCLUZII

Cu această lucrare am făcut primii pași reali în Python și am parcurs pe rând patru părți: „Hello World” și „Variables” pe [codedex.io](https://www.codedex.io), apoi tutorialele de bază și cele avansate de pe [learnpython.org](https://www.learnpython.org). Am început cu lucruri mici, precum configurarea mediului, primul program „Hello World” și exerciții simple de afișare. Odată ce le-am prins, mi-a fost mult mai ușor să trec la următorul nivel.

Partea cu variabilele a fost prima în care am făcut ceva util. Am lucrat cu tipuri de date și am scris mici programe pentru conversia temperaturii, calculul indicelui de masă corporală (BMI), teorema lui Pitagora și schimbul valutar. Pe [learnpython.org](https://www.learnpython.org) am acoperit fundamentele: liste, operatori, șiruri de caractere, condiții, bucle, funcții, clase și obiecte, dicționare, module și pachete. Cele mai multe lucruri le-am înțeles abia când le-am scris singur, nu când le-am citit.

Tutorialele avansate au fost mai grele, dar și mai interesante. Generatoarele, list comprehensions, funcțiile lambda, expresiile regulate, excepțiile, mulțimile, serializarea, closures, decoratorii și ``map`/`filter`/`reduce`` mi-au arătat că Python poate face în puține rânduri ce în alte limbaje ar cere mult mai mult cod. Unele concepte, cum sunt decoratorii și closures, au cerut mai mult timp și mai multe încercări până să le pricep.

Acum mă simt mult mai confortabil cu Python decât la început. Știu să scriu programe simple, să folosesc structurile de date și să înțeleg cod scris de alții. Pentru Tehnici de programare și pentru proiectele viitoare, cred că asta e o bază bună, iar restul o să vină cu practică.

BIBLIOGRAFIE

1. Codedex.io - <https://www.codedex.io/python>
2. Learnpython - <https://www.learnpython.org/>